Multiheaded attention and its gradient

Siavash Ahmadi

Below, I'll be repeating the same idea multiple times, progressively adding bells and whistles to hopefully make this easier to follow. The calculations are presented in tensor (N-D array) format to make them directly compatible with NumPy, with N referring to the number of batches.

1 Attention

NOTE: Don't be intimidated by attention! If right off the bat you want to see a single equation that fully describes multihead attention, check out Eq. (4)*. Also see Figure 1!*

The core of the Transformer architecture is the attention function, which is surprisingly simple and straightforward to describe. Attention is a function of three inputs arguments (which are matrices) and one output argument (which is a matrix), and can be separated into three semantically distinct steps:

1. In-projections
2. Scaled dot-product attention
3. Out-projection

Note that these three steps are just names we're using to partition the operations of the attention function in an easier to digest way. Attention is just the composition of several function. These the names refer to the following equations:

1. In-projections

$$\mathbf{Q} = \mathbf{q}\mathbf{W}^{\mathbf{q}}, \quad \mathbf{K} = \mathbf{k}\mathbf{W}^{\mathbf{k}}, \quad \mathbf{V} = \mathbf{v}\mathbf{W}^{\mathbf{v}}$$

   Here, the $\mathbf{W}$ superscripts little $^{\mathbf{q}}$, little $^{\mathbf{k}}$, and little $^{\mathbf{v}}$ are not indexes—they are just part of the variable's name.

2. Scaled dot-product attention

$$\mathbf{A} = \text{softmax}(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}})\mathbf{V}$$

   where d_k is a scalar hyperparameter we will get to later.

3. Out-projection

$$\mathbf{Z} = \mathbf{A}\mathbf{W}^{\mathbf{o}}$$

   Here, too, the superscript little $^{\mathbf{o}}$ is part of the variable's name and is not an index.

2 Multiheaded Attention

The transformer uses multiheaded attention which is just a fancy-sounding way to say it repeats the above operations multiple times using different values for the four $\mathbf{W}$ parameter matrices.

The transformer multiheaded attention boils down to the following equations, where we now use the $i = 1, \ldots, h$ subscript *indexes* to distinguish between the different *heads* and write the whole thing a bit more formally:

$$\mathbf{Q}_i = \mathbf{q}\mathbf{W}_i^{\mathbf{q}}, \quad \mathbf{K}_i = \mathbf{k}\mathbf{W}_i^{\mathbf{k}}, \quad \mathbf{V}_i = \mathbf{v}\mathbf{W}_i^{\mathbf{v}} \tag{1}$$

$$\mathbf{A}_i = \text{softmax}(\frac{\mathbf{Q}_i\mathbf{K}_i^\top}{\sqrt{d_k}})\mathbf{V}_i \tag{2}$$

$$\mathbf{Z} = \begin{bmatrix}\mathbf{A}_1 & \mathbf{A}_2 & \cdots & \mathbf{A}_h\end{bmatrix}\mathbf{W}^{\mathbf{o}} \tag{3}$$

Putting it all together we get the following (multiheaded attention in a single equation):

$$\mathbf{Z}(\mathbf{q},\mathbf{k},\mathbf{v}) = \begin{bmatrix} \text{softmax}(\frac{\mathbf{q}\mathbf{W}_1^{\mathbf{q}}(\mathbf{k}\mathbf{W}_1^{\mathbf{k}})^\top}{\sqrt{d_k}})\mathbf{v}\mathbf{W}_1^{\mathbf{v}} \\ \text{softmax}(\frac{\mathbf{q}\mathbf{W}_2^{\mathbf{q}}(\mathbf{k}\mathbf{W}_2^{\mathbf{k}})^\top}{\sqrt{d_k}})\mathbf{v}\mathbf{W}_2^{\mathbf{v}} \\ \vdots \\ \text{softmax}(\frac{\mathbf{q}\mathbf{W}_h^{\mathbf{q}}(\mathbf{k}\mathbf{W}_h^{\mathbf{k}})^\top}{\sqrt{d_k}})\mathbf{v}\mathbf{W}_h^{\mathbf{v}} \end{bmatrix}^\top \mathbf{W}^{\mathbf{o}} \tag{4}$$

where the three-input $\mathbf{Z}$ (the multiheaded attention with h heads) is parameterized by $3h + 1$ matrices: $\mathbf{W}_i{}^{\mathbf{q}}$, $\mathbf{W}_i{}^{\mathbf{k}}$, $\mathbf{W}_i{}^{\mathbf{v}}$, and $\mathbf{W}^{\mathbf{o}}$, $i = 1, \ldots, h$.

The "hardest" thing about implementing multiheaded attention from scratch is managing the matrix shapes effectively so as to make the matrix multiplications efficient.

3 Full description of multiheaded attention operations

3.1 In projections

Inputs	Parameters	Forward pass operations	Outputs
$\mathbf{q} \in \mathbb{R}^{N\times L_q\times d_{\text{model}}}$	$\mathbf{W_q} \in \mathbb{R}^{h\times d_{\text{model}}\times d_k}$	`Q = np.dot(q,Wq)` `Q = np.moveaxis(Q, -2, 0)`	$\mathbf{Q} \in \mathbb{R}^{h\times N\times L_q\times d_k}$
$\mathbf{k} \in \mathbb{R}^{N\times L_k\times kdim}$	$\mathbf{W_k} \in \mathbb{R}^{h\times kdim\times d_k}$	`K = np.dot(k,Wk)` `K = np.moveaxis(K, -2, 0)`	$\mathbf{K} \in \mathbb{R}^{h\times N\times L_k\times d_k}$
$\mathbf{v} \in \mathbb{R}^{N\times L_k\times vdim}$	$\mathbf{W_v} \in \mathbb{R}^{h\times vdim\times d_v}$	`V = np.dot(v,Wv)` `V = np.moveaxis(V, -2, 0)`	$\mathbf{V} \in \mathbb{R}^{h\times N\times L_k\times d_v}$

Here, $d_{\text{model}} = hd_k$ is the *dimensionality of the model*, i.e., the dimensionality of the token embeddings coming in. We typically choose d_{model} and h and calculate d_k as $\frac{d_{\text{model}}}{h}$, implying we require d_{model} to be divisible by h.

3.2 Scaled dot product attention

3.2.1 Scaled dot product

Inputs	Forward pass operations	Outputs
$\mathbf{Q} \in \mathbb{R}^{h \times N \times L_q \times d_k}$ $\mathbf{K} \in \mathbb{R}^{h \times N \times L_k \times d_k}$	`s = 1 / np.sqrt(d_k)` `S = Q @ np.swapaxes(K, -1, -2) * s`	$\mathbf{S} \in \mathbb{R}^{h \times N \times L_q \times L_k}$

3.2.2 Masked Softmax

For the softmax gradient, see softmax and its gradient (PDF).

Inputs	Forward pass operations	Outputs
$\mathbf{S} \in \mathbb{R}^{h \times N \times L_q \times L_k}$	See softmax and its gradient PDF	$\mathbf{A} \in \mathbb{R}^{h \times N \times L_q \times L_k}$

3.2.3 Attention-weighted values

Inputs	Forward pass operations	Outputs
$\mathbf{A} \in \mathbb{R}^{h \times N \times L_q \times L_k}$ $\mathbf{V} \in \mathbb{R}^{h \times N \times L_k \times d_v}$	`a = A @ V # shape (h, N, L_q, d_v)` `a' = np.concatenate(a, axis=-1)`	$\mathbf{a}' \in \mathbb{R}^{N \times L_q \times hd_v}$

We used a different name $\mathbf{a}'$ for the concatenated form of $\mathbf{a}$ because later this will make reasoning about the gradients clearer.

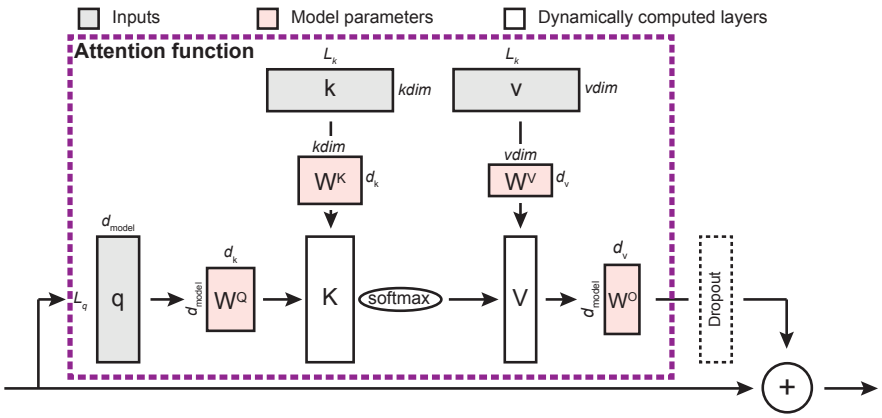

Figure 1: **Attention function and the residual stream**. This figure visually describes the function discussed in this document. It also shows the residual stream (bottom arrow and the summation $\oplus$) and the (optional) dropout layer. Note that this representation obfuscates some intermediate steps which have been made explicit in the equations of this document.

3.3 Out projection

Inputs	Parameters	Forward pass operations	Outputs
$\mathbf{a}' \in \mathbb{R}^{N \times L_q \times hd_v}$	$\mathbf{W_o} \in \mathbb{R}^{hd_v \times d_{\text{model}}}$	`Z = a' @ Wo`	$\mathbf{Z} \in \mathbb{R}^{N \times L_q \times d_{\text{model}}}$

Note that the output $\mathbf{Z}$ has the exact same dimensionality as the input $\mathbf{q}$, and this is by design, which is why we can add the residual stream and stack multiple Multiheaded Attention blocks sequentially. To see this graphically for a single head, see Figure 1.

4 The gradients (backward pass)

We begin deriving the gradients from the last operation, the out projection, and work our way back to the inputs. For any operation $y = f(x)$, we will use the shorthand

$$G = \frac{\partial \mathcal{L}}{\partial y} \in \mathcal{X}$$

for the gradient of the final loss function $\mathcal{L}$ with respect to the current operation's output y. $\mathcal{X}$ is the set to which both G and y belong (determines dimensionality of both). We refer to G as the "upstream gradient."

4.1 Out projection

$$\mathbf{Z} = \mathbf{a}'\mathbf{W^o} = \begin{bmatrix}\mathbf{a}_1 & \mathbf{a}_2 & \cdots & \mathbf{a}_h\end{bmatrix} \mathbf{W^o} \in \mathbb{R}^{N \times L_q \times d_{\text{model}}} \qquad \text{(forward pass)}$$

$$\frac{\partial \mathbf{Z}}{\partial \mathbf{a}'} = \mathbf{W^o}^\top \in \mathbb{R}^{d_{\text{model}} \times hd_v}, \quad \frac{\partial \mathbf{Z}}{\partial \mathbf{W^o}} = \mathbf{a}'^\top \in \mathbb{R}^{N \times hd_v \times L_q} \qquad \text{(local gradients)}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}'} = G\frac{\partial \mathbf{Z}}{\partial \mathbf{a}'} \in \mathbb{R}^{N \times L_q \times hd_v}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{W^o}} = \frac{\partial \mathbf{Z}}{\partial \mathbf{W^o}}G \in \mathbb{R}^{N \times hd_v \times d_{\text{model}}} \qquad \text{(upstream of lower operations)}$$

To match the dimensionality of $\frac{\partial \mathcal{L}}{\partial \mathbf{W^o}}$ with that of $\mathbf{W^o}$, we must sum over the batch dimension[1]:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W^o}} = \sum_{b \in \text{batch}} \left(\frac{\partial \mathbf{Z}}{\partial \mathbf{W^o}}G\right)_b \in \mathbb{R}^{hd_v \times d_{\text{model}}}$$

We also have

$$\begin{aligned}\frac{\partial \mathbf{Z}}{\partial \mathbf{a}'} &= \begin{bmatrix}\mathbf{W^o}_1{}^\top & \mathbf{W^o}_2{}^\top & \cdots & \mathbf{W^o}_h{}^\top\end{bmatrix} \\ &= \begin{bmatrix}\frac{\partial \mathbf{Z}}{\partial \mathbf{a}_1} & \frac{\partial \mathbf{Z}}{\partial \mathbf{a}_2} & \cdots & \frac{\partial \mathbf{Z}}{\partial \mathbf{a}_h}\end{bmatrix}\end{aligned}$$

where we rewrote $\mathbf{W^o}$ as

[1]This is not the *real* reason but it sounds like a better mnemonic. The real reason is that by performing the preceding calculations, we have computed N different gradients as a result of the N batches, and that, by chain rule, gradients flowing in to a parameter from multiple downstream computations sum at the parameter.

$$\mathbf{W^o} = \begin{bmatrix} \mathbf{W^o}_1 & \mathbf{W^o}_2 & \cdots & \mathbf{W^o}_h \end{bmatrix}^\top, \quad \mathbf{W^o}_i \in \mathbb{R}^{d_v \times d_{\text{model}}}$$

Now, since

$$\mathbf{a} = \begin{bmatrix} [\mathbf{a}_1] & [\mathbf{a}_2] & \cdots & [\mathbf{a}_h] \end{bmatrix} \in \mathbb{R}^{h \times N \times L_q \times d_v} \ (\mathbf{a}_i \in \mathbb{R}^{N \times L_q \times d_v})$$

we have

$$\begin{aligned} \frac{\partial \mathbf{Z}}{\partial \mathbf{a}} &= \begin{bmatrix} [\frac{\partial \mathbf{Z}}{\partial \mathbf{a}_1}] & [\frac{\partial \mathbf{Z}}{\partial \mathbf{a}_2}] & \cdots & [\frac{\partial \mathbf{Z}}{\partial \mathbf{a}_h}] \end{bmatrix} \\ &= \begin{bmatrix} [\mathbf{W^o}_1{}^\top] & [\mathbf{W^o}_2{}^\top] & \cdots & [\mathbf{W^o}_h{}^\top] \end{bmatrix} \end{aligned}$$

Finally, we can compute the gradient of the loss with respect to $\mathbf{a}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}} = G \frac{\partial \mathbf{Z}}{\partial \mathbf{a}} \in \mathbb{R}^{N \times L_q \times h \times d_v}$$

which we compute in NumPy as follows

```
grad["dZ_da'"] = np.moveaxis(self.Wo, -1, -2) # (d_model, d_v * h)
grad["dZ_da"]  = np.stack(
                    np.split(grad["dZ_da'"], num_heads, axis=-1)
                 ) # (h, d_model, d_v)
grad["dL_da"] = np.dot(upstream_grad, grad["dZ_da"]) # (N, L_q, h, d_v)
                                                     #   = (N, L_q, d_model) . (h, d_model, d_v)
grad["dL_da"] = np.moveaxis(grad["dL_da"], -2, 0) # (h, N, L_q, d_v)
```

In the last equation, $G = \frac{\partial \mathcal{L}}{\partial \mathbf{Z}}$ is the upstream gradient.

4.2 Scaled dot product attention

4.2.1 Attention-weighted values

$$\mathbf{a} = \mathbf{AV} \in \mathbb{R}^{h \times N \times L_q \times d_v} \qquad \text{(forward pass)}$$

$$\frac{\partial \mathbf{a}}{\partial \mathbf{A}} = \mathbf{V}^\top \in \mathbb{R}^{h \times N \times d_v \times L_k}, \quad \frac{\partial \mathbf{a}}{\partial \mathbf{V}} = \mathbf{A}^\top \in \mathbb{R}^{h \times N \times L_k \times L_q} \qquad \text{(local gradients)}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{A}} = G \frac{\partial \mathbf{a}}{\partial \mathbf{A}} \in \mathbb{R}^{h \times N \times L_q \times L_k}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{V}} = \frac{\partial \mathbf{a}}{\partial \mathbf{V}} G \in \mathbb{R}^{h \times N \times L_k \times d_v} \qquad \text{(upstream of lower operations)}$$

where $G = \frac{\partial \mathcal{L}}{\partial \mathbf{a}}$ is the upstream gradient.

4.2.2 Masked softmax

Please see the PDF on the softmax function and its gradient. As a reminder, here's the gradient with respect to the masked softmax inputs of the loss:

$$\mathbf{A} = \text{softmax}(\mathbf{S}) \qquad \text{(forward pass)}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}} = (G - G \cdot \mathbf{p}) \odot \mathbf{p} \in \mathbb{R}^{h \times N \times L_q \times d_v} \qquad \text{(upstream of lower operations)}$$

4.2.3 Scaled dot product

$$s = \frac{1}{\sqrt{d_K}}, \quad \mathbf{S} = s\mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{h \times N \times L_q \times L_k} \qquad \text{(forward pass)}$$

$$\frac{\partial \mathbf{S}}{\partial \mathbf{Q}} = s\mathbf{K} \in \mathbb{R}^{h \times N \times L_k \times d_k}, \quad \frac{\partial \mathbf{S}}{\partial \mathbf{K}} = s\mathbf{Q} \in \mathbb{R}^{h \times N \times L_q \times d_k} \qquad \text{(local gradients)}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{Q}} = sG\mathbf{K} \in \mathbb{R}^{h \times N \times L_q \times d_k}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{K}} = sG^\top \mathbf{Q} \in \mathbb{R}^{h \times N \times L_k \times d_k}$$

(upstream of lower operations)

4.3 In projections

For each head i, we have:

$$\mathbf{Q}_i = \mathbf{q}\mathbf{W}_i^{\mathbf{q}}, \quad \mathbf{K}_i = \mathbf{k}\mathbf{W}_i^{\mathbf{k}}, \quad \mathbf{V}_i = \mathbf{v}\mathbf{W}_i^{\mathbf{v}} \qquad \text{(forward pass)}$$

$$\frac{\partial \mathbf{Q}_i}{\partial \mathbf{q}} = \mathbf{W}_i^{\mathbf{q}\top}, \quad \frac{\partial \mathbf{K}_i}{\partial \mathbf{k}} = \mathbf{W}_i^{\mathbf{k}\top}, \quad \frac{\partial \mathbf{V}_i}{\partial \mathbf{v}} = \mathbf{W}_i^{\mathbf{v}\top} \qquad \text{(local gradients)}$$

$$\frac{\partial \mathbf{Q}_i}{\partial \mathbf{W}_i^{\mathbf{q}}} = \mathbf{q}^\top, \quad \frac{\partial \mathbf{K}_i}{\partial \mathbf{W}_i^{\mathbf{k}}} = \mathbf{k}^\top, \quad \frac{\partial \mathbf{V}_i}{\partial \mathbf{W}_i^{\mathbf{v}}} = \mathbf{v}^\top \qquad \text{(local gradients)}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{q}} = G\frac{\partial \mathbf{Q}_i}{\partial \mathbf{q}}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{k}} = G\frac{\partial \mathbf{Q}_i}{\partial \mathbf{q}}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{v}} = G\frac{\partial \mathbf{Q}_i}{\partial \mathbf{q}} \qquad \text{(upstream of lower operations)}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_i^{\mathbf{q}}} = \frac{\partial \mathbf{Q}_i}{\partial \mathbf{W}_i^{\mathbf{q}}}G, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{W}_i^{\mathbf{k}}} = \frac{\partial \mathbf{K}_i}{\partial \mathbf{W}_i^{\mathbf{k}}}G, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{W}_i^{\mathbf{v}}} = \frac{\partial \mathbf{V}_i}{\partial \mathbf{W}_i^{\mathbf{v}}}G \qquad \text{(upstream of lower operations)}$$

Now, to pack all of this into vectorized NumPy operations, we rewrite everything without the subscripts while being very careful about the shapes. We derive this for $\mathbf{q}$ and $\mathbf{W}^{\mathbf{q}}$ only as the equations for the rest are similar.

$$\mathbf{Q}' = \mathbf{q}\mathbf{W}^{\mathbf{q}} \in \mathbb{R}^{N \times L_q \times h \times d_k} \qquad \text{(forward pass)}$$

$$\mathbf{Q} = \texttt{np.moveaxis}(\mathbf{Q}', -2, 0) \in \mathbb{R}^{h \times N \times L_q \times d_k} \ (\ni G) \qquad \text{(forward pass)}$$

$$\frac{\partial \mathbf{Q}}{\partial \mathbf{q}} = \mathbf{W}^{\mathbf{q}\top} \in \mathbb{R}^{h \times d_k \times d_{\text{model}}}, \quad \frac{\partial \mathbf{Q}}{\partial \mathbf{W}^{\mathbf{q}}} = \mathbf{q}^\top \in \mathbb{R}^{N \times d_{\text{model}} \times L_q} \qquad \text{(local gradients)}$$

Here comes the tricky part: in the per-head equation we saw that $\frac{\partial \mathcal{L}}{\partial \mathbf{q}} = G \frac{\partial \mathbf{Q}_i}{\partial \mathbf{q}}$, but this will not give the correct shape if we simply carry out the product as follows:

```
grad["dL_dq"] = G @ grad["dQ_dq"] # (h, N, L_q, d_k) @ (h, d_k, d_model)
                                  #  won't run; shapes don't match!
```

the `@` operator won't complete properly because after it lines up the tensors on the last two axes ($L_q \times d_k$ and $d_k \times d_{\text{model}}$), the remaining axes are neither identical nor broadcastable (expected both to be either $h \times N$ or h or N, but got $h \times N$ vs. h). Instead, we should be adding an axis to allow NumPy's broadcasting rules to kick in:

$$\frac{\partial \mathbf{Q}}{\partial \mathbf{q}} = \mathbf{W}^{\mathbf{q}\top} \in \mathbb{R}^{h \times 1 \times d_k \times d_{\text{model}}}, \quad \frac{\partial \mathbf{Q}}{\partial \mathbf{W}^{\mathbf{q}}} = \mathbf{q}^\top \in \mathbb{R}^{1 \times N \times d_{\text{model}} \times L_q} \qquad \text{(local gradients)}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{q}} = G \frac{\partial \mathbf{Q}}{\partial \mathbf{q}} \in \mathbb{R}^{h \times N \times L_q \times d_{\text{model}}}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{\mathbf{q}}} = \frac{\partial \mathbf{Q}}{\partial \mathbf{W}^{\mathbf{q}}} G \in \mathbb{R}^{h \times N \times d_{\text{model}} \times d_k}$$

(upstream of lower operations)

This makes it so that the matrices represented by the final two dimensions interact independently for each head and batch.

Finally to make sure the dimensions are correct, we should be summing along the batch axis:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{q}} = \sum_{b \in \text{batch}} \left(G \frac{\partial \mathbf{Q}}{\partial \mathbf{q}} \right)_b \in \mathbb{R}^{h \times L_q \times d_{\text{model}}}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{\mathbf{q}}} = \sum_{b \in \text{batch}} \left(\frac{\partial \mathbf{Q}}{\partial \mathbf{W}^{\mathbf{q}}} G \right)_b \in \mathbb{R}^{h \times d_{\text{model}} \times d_k}$$

Similar arguments hold for the other matrices.